

CS 348: Module 4

Relational Algebra

Spring 2026

Reading Assignments and References

To be read during the Week of June 1–5:[†]

- ▶ Chapter 2, Sections 2.3 and 2.6.

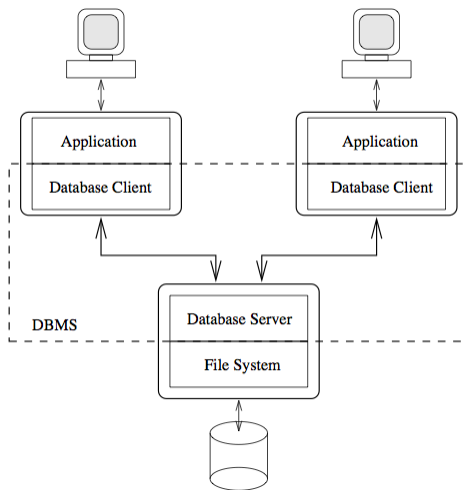
[†]7th edition of course textbook: Avi Silberschatz, Henry F. Korth, and S. Sudarshan
Database System Concepts.

Outline

Unit 1: **Overview**

Unit 2: Relational Algebra

The Two-Tier Architecture



Also called the **client/server architecture**.

The Two-Tier Architecture (cont'd)

Application:

- ▶ User interaction: query input, presentation of results.
- ▶ Application-specific tasks.

Database Server:

- ▶ DDL evaluation.
- ▶ DML compilation: *selection of a query plan for a query.*
- ▶ DML execution.
- ▶ Concurrency control.
- ▶ Buffer management: rollback and failure recovery.

File System:

- ▶ Storage and retrieval of unstructured data.

Query Evaluation

Steps in evaluating a query Q :

1. Parsing, view expansion, and type and authorization checking of Q .
2. Translation of Q to a formulation E_Q in the **relational algebra**.
3. Optimization of E_Q .
 - $\Rightarrow$ generates an efficient **query plan** P_Q from E_Q
 - $\Rightarrow$ uses statistical metadata about the database instance
4. Execution of query plan P_Q .
 - $\Rightarrow$ uses **access methods** to access stored relations
 - $\Rightarrow$ uses **physical relational operators** to combine relations

Considerations:

- ▶ How relations are stored, that is, *physically represented*.
- ▶ Choice of physical relational operators to answer to complex queries.
- ▶ How *intermediate results* are managed.

Outline

Unit 1: Overview

Unit 2: **Relational Algebra**

Relational Algebra: Overview

Idea

Define a **relational algebra (RA)** consisting of a **set of operations** on the universe $\mathcal{U}$ of finite relations over an underlying universe of values $\mathbf{D}$ of a database instance **DB**.

$$(\mathcal{U}; R_0, \dots, R_k, \times, \sigma, \pi, \rho, \cup, -, A = c_1, A = c_2, \dots)$$

Constants:

R_i **relation name** (*one for each relation name in a signature ρ*)

$A = c_i$ **constant** (*one for each constant in $\mathbf{D}$*)

Unary operators:

σ **selection** (*removes rows*)

π **projection** (*removes columns and duplicates*)

ρ **rename**

Binary operators:

$\times$ **cross product**

$\cup$ **union**

$-$ **set difference**

Relational Algebra: Syntax

Definition

Given a database signature ρ , where table arity is defined by a sequence of attributes, and a set of constants $\{c_1, c_2, \dots\}$, a *relational algebra* (RA) query is an expression E given as follows:

$$\begin{array}{lcl} E & ::= & R_i \\ & | & A = c_j \\ & | & \sigma_{A_i=A_j}(E_1) \\ & | & \pi_{(A_1, \dots, A_k)}(E_1) \\ & | & \rho_{(A_1, \dots, A_k)}(E_1) \\ & | & E_1 \times E_2 \\ & | & E_1 \cup E_2 \\ & | & E_1 - E_2 \end{array}$$

We write $\text{Eval}(E, \mathbf{DB})$ to denote the table computed by evaluating the RA query E and $\text{Attr}(E)$ to denote its sequence of *distinct* attributes $(A_1, \dots, A_k)$.

Relational Algebra: Examples

(signature) $\rho = ($
 ACCOUNT(anum, type, balance, bank, bnum),
 BANK(name, address))

(data) **DB** = (STR $\uplus$ Z, $\approx$,

ACCOUNT =

anum	type	balance	bank	bnum
1234	CHK	\$1000	TD	1
1235	SAV	\$20000	TD	2
1236	CHK	\$2500	CIBC	1
1237	CHK	\$2500	Royal	5
2000	BUS	\$10000	Royal	5
2001	BUS	\$10000	TD	3

,

BANK =

name	address
TD	TD Centre
CIBC	CIBC Tower

)

Relational Algebra: Examples (cont'd)

relation name

Example: *All account information.*

Eval(**ACCOUNT**, **DB**) =

anum	type	balance	bank	bnun
1234	CHK	\$1000	TD	1
1235	SAV	\$20000	TD	2
1236	CHK	\$2500	CIBC	1
1237	CHK	\$2500	Royal	5
2000	BUS	\$10000	Royal	5
2001	BUS	\$10000	TD	3

Relational Algebra: Examples (cont'd)

relation name

Example: *All account information.*

$\text{Eval}(\text{ACCOUNT}, \mathbf{DB}) =$

anum	type	balance	bank	bnum
1234	CHK	\$1000	TD	1
1235	SAV	\$20000	TD	2
1236	CHK	\$2500	CIBC	1
1237	CHK	\$2500	Royal	5
2000	BUS	\$10000	Royal	5
2001	BUS	\$10000	TD	3

constant

Example: \$10000.

$\text{Eval}(\text{amount} = \$10000, \mathbf{DB}) =$

amount
\$10000

Relational Algebra: Examples (cont'd)

cross product

Example: *Every pair of accounts and banks.*

Eval($\text{ACCOUNT} \times \text{BANK}$, **DB**)

=

anum	type	balance	bank	bnum	name	address
1234	CHK	\$1000	TD	1	TD	TD Centre
1234	CHK	\$1000	TD	1	CIBC	CIBC Tower
1235	SAV	\$20000	TD	2	TD	TD Centre
1235	SAV	\$20000	TD	2	CIBC	CIBC Tower
1236	CHK	\$2500	CIBC	1	TD	TD Centre
1236	CHK	\$2500	CIBC	1	CIBC	CIBC Tower
1237	CHK	\$2500	Royal	5	TD	TD Centre
1237	CHK	\$2500	Royal	5	CIBC	CIBC Tower
2000	BUS	\$10000	Royal	5	TD	TD Centre
2000	BUS	\$10000	Royal	5	CIBC	CIBC Tower
2001	BUS	\$10000	TD	3	TD	TD Centre
2001	BUS	\$10000	TD	3	CIBC	CIBC Tower

Relational Algebra: Examples (cont'd)

selection

Example: *All account information, including bank addresses.*

Eval($\sigma_{\text{bank} = \text{name}}$ (ACCOUNT $\times$ BANK), **DB**)

=

anum	type	balance	bank	bnum	name	address
1234	CHK	\$1000	TD	1	TD	TD Centre
1235	SAV	\$20000	TD	2	TD	TD Centre
1236	CHK	\$2500	CIBC	1	CIBC	CIBC Tower
2001	BUS	\$10000	TD	3	TD	TD Centre

Relational Algebra: Examples (cont'd)

selection

Example: *All account information, including bank addresses.*

Eval($\sigma_{\text{bank} = \text{name}}$ (ACCOUNT $\times$ BANK), **DB**)

=

anum	type	balance	bank	bnum	name	address
1234	CHK	\$1000	TD	1	TD	TD Centre
1235	SAV	\$20000	TD	2	TD	TD Centre
1236	CHK	\$2500	CIBC	1	CIBC	CIBC Tower
2001	BUS	\$10000	TD	3	TD	TD Centre

Example: *All account information for accounts with a \$10000 balance.*

Eval($\sigma_{\text{balance} = \text{amount}}$ (ACCOUNT $\times$ amount = \$10000), **DB**)

=

anum	type	balance	bank	bnum	amount
2000	BUS	\$10000	Royal	5	\$10000
2001	BUS	\$10000	TD	3	\$10000

Relational Algebra: Examples (cont'd)

projection

Example: *The type and balance of all accounts.*

$\text{Eval}(\pi_{(\text{type}, \text{balance})}(\text{ACCOUNT}), \mathbf{DB}) =$

type	balance
CHK	\$1000
SAV	\$20000
CHK	\$2500
BUS	\$10000

Relational Algebra: Examples (cont'd)

projection

Example: *The type and balance of all accounts.*

$$\text{Eval}(\pi_{(\text{type}, \text{balance})}(\text{ACCOUNT}), \mathbf{DB}) =$$

type	balance
CHK	\$1000
SAV	\$20000
CHK	\$2500
BUS	\$10000

rename

Example: *Renaming bank attributes.*

$$\text{Eval}(\rho_{(\text{bname}, \text{addr})}(\text{BANK}), \mathbf{DB}) =$$

bname	addr
TD	TD Centre
CIBC	CIBC Tower

Relational Algebra: Examples (cont'd)

union

Example: *The type and balance of all checking and savings accounts.*

Eval($\rho_{(\text{type}, \text{balance})}(\sigma_{\text{type}=\text{t}}(\text{ACCOUNT} \times ((\text{t} = \text{CHK}) \cup (\text{t} = \text{SAV}))))$), **DB**)

=

type	balance
CHK	\$1000
SAV	\$20000
CHK	\$2500

Relational Algebra: Examples (cont'd)

union

Example: *The type and balance of all checking and savings accounts.*

$\text{Eval}(\rho_{(\text{type}, \text{balance})} (\sigma_{\text{type}=\text{t}} (\text{ACCOUNT} \times ((\text{t} = \text{CHK}) \cup (\text{t} = \text{SAV}))))), \mathbf{DB})$

=

type	balance
CHK	\$1000
SAV	\$20000
CHK	\$2500

difference

Example: *Banks that do not have addresses.*

$\text{Eval}(\pi_{(\text{bank})} (\text{ACCOUNT}) - \rho_{(\text{bank})} (\pi_{(\text{name})} (\text{BANK})), \mathbf{DB})$

bank
Royal

Semantics: A Mapping to Range Restricted RC

- ▶ The semantics of $\text{Eval}(E, \mathbf{DB})$ is given indirectly by defining a mapping to the range restricted relational calculus defined in Module 2.
- ▶ We also assume that attribute names can also qualify as variable names.

Semantics: A Mapping to Range Restricted RC

- ▶ The semantics of $\text{Eval}(E, \mathbf{DB})$ is given indirectly by defining a mapping to the range restricted relational calculus defined in Module 2.
- ▶ We also assume that attribute names can also qualify as variable names.

relation name (for $R_i / (A_1, \dots, A_k) \in \rho$)

$$\text{RAtoRC}(R_i) = \{(A_1, \dots, A_k) \mid R_i(A_1, \dots, A_k)\}.$$

Semantics: A Mapping to Range Restricted RC

- ▶ The semantics of $\text{Eval}(E, \mathbf{DB})$ is given indirectly by defining a mapping to the range restricted relational calculus defined in Module 2.
- ▶ We also assume that attribute names can also qualify as variable names.

relation name (for $R_i / (A_1, \dots, A_k) \in \rho$)

$$\text{RAtoRC}(R_i) = \{(A_1, \dots, A_k) \mid R_i(A_1, \dots, A_k)\}.$$

constant

$$\text{RAtoRC}(A = c) = \{(A) \mid A = c\}.$$

Semantics (cont'd)

selection

$$\begin{aligned} \text{RAtRC}(\sigma_{A_i=A_j}(E_1)) &= \{(A_1, \dots, A_k) \mid \varphi \wedge (A_i = A_j)\}, \\ \text{where } \text{RAtRC}(E_1) &= \{(A_1, \dots, A_k) \mid \varphi\}. \end{aligned}$$

Semantics (cont'd)

selection

$$\text{RAtRC}(\sigma_{A_i=A_j}(E_1)) = \{(A_1, \dots, A_k) \mid \varphi \wedge (A_i = A_j)\},$$

where $\text{RAtRC}(E_1) = \{(A_1, \dots, A_k) \mid \varphi\}$.

projection

$$\text{RAtRC}(\pi_{(A_{i_1}, \dots, A_{i_k})}(E_1)) = \{(A_{i_1}, \dots, A_{i_k}) \mid \exists A_{i_{k+1}} \dots \exists A_{i_n} \cdot \varphi\},$$

where $\text{RAtRC}(E_1) = \{(A_1, \dots, A_n) \mid \varphi\}$
and where $\{i_1, \dots, i_n\} = \{1, \dots, n\}$.

Semantics (cont'd)

selection

$$\text{RAtRC}(\sigma_{A_i=A_j}(E_1)) = \{(A_1, \dots, A_k) \mid \varphi \wedge (A_i = A_j)\},$$

where $\text{RAtRC}(E_1) = \{(A_1, \dots, A_k) \mid \varphi\}$.

projection

$$\text{RAtRC}(\pi_{(A_{i_1}, \dots, A_{i_k})}(E_1)) = \{(A_{i_1}, \dots, A_{i_k}) \mid \exists A_{i_{k+1}} \dots \exists A_{i_n} \cdot \varphi\},$$

where $\text{RAtRC}(E_1) = \{(A_1, \dots, A_n) \mid \varphi\}$
and where $\{i_1, \dots, i_n\} = \{1, \dots, n\}$.

rename

$$\text{RAtRC}(\rho_{(A_1, \dots, A_k)}(E_1)) = \{(A_1, \dots, A_k) \mid \varphi[A_1/B_1] \cdots [A_k/B_k]\},$$

where $\text{RAtRC}(E_1) = \{(B_1, \dots, B_k) \mid \varphi\}$
and where $\varphi[A_i/B_i]$ substitutes all free occurrences of B_i in φ with A_i .

Semantics (cont'd)

cross product

$$\text{RAtRC}(E_1 \times E_2) = \{(A_1, \dots, A_m, B_1, \dots, B_n) \mid \varphi_1 \wedge \varphi_2\},$$

$$\text{where } \text{RAtRC}(E_1) = \{(A_1, \dots, A_m) \mid \varphi_1\}$$

$$\text{and where } \text{RAtRC}(E_2) = \{(B_1, \dots, B_n) \mid \varphi_2\}.$$

Semantics (cont'd)

cross product

$$\begin{aligned}\text{RAtRC}(E_1 \times E_2) &= \{(A_1, \dots, A_m, B_1, \dots, B_n) \mid \varphi_1 \wedge \varphi_2\}, \\ \text{where } \text{RAtRC}(E_1) &= \{(A_1, \dots, A_m) \mid \varphi_1\} \\ \text{and where } \text{RAtRC}(E_2) &= \{(B_1, \dots, B_n) \mid \varphi_2\}.\end{aligned}$$

union

$$\begin{aligned}\text{RAtRC}(E_1 \cup E_2, \mathbf{DB}) &= \{(A_1, \dots, A_k) \mid \varphi_1 \vee \varphi_2\}, \\ \text{where } \text{RAtRC}(E_1) &= \{(A_1, \dots, A_k) \mid \varphi_1\} \\ \text{and where } \text{RAtRC}(E_2) &= \{(A_1, \dots, A_k) \mid \varphi_2\}.\end{aligned}$$

Semantics (cont'd)

cross product

$$\begin{aligned}\text{RAtoRC}(E_1 \times E_2) &= \{(A_1, \dots, A_m, B_1, \dots, B_n) \mid \varphi_1 \wedge \varphi_2\}, \\ \text{where } \text{RAtoRC}(E_1) &= \{(A_1, \dots, A_m) \mid \varphi_1\} \\ \text{and where } \text{RAtoRC}(E_2) &= \{(B_1, \dots, B_n) \mid \varphi_2\}.\end{aligned}$$

union

$$\begin{aligned}\text{RAtoRC}(E_1 \cup E_2, \mathbf{DB}) &= \{(A_1, \dots, A_k) \mid \varphi_1 \vee \varphi_2\}, \\ \text{where } \text{RAtoRC}(E_1) &= \{(A_1, \dots, A_k) \mid \varphi_1\} \\ \text{and where } \text{RAtoRC}(E_2) &= \{(A_1, \dots, A_k) \mid \varphi_2\}.\end{aligned}$$

set difference

$$\begin{aligned}\text{RAtoRC}(E_1 - E_2, \mathbf{DB}) &= \{(A_1, \dots, A_k) \mid \varphi_1 \wedge \neg \varphi_2\}, \\ \text{where } \text{RAtoRC}(E_1) &= \{(A_1, \dots, A_k) \mid \varphi_1\} \\ \text{and where } \text{RAtoRC}(E_2) &= \{(A_1, \dots, A_k) \mid \varphi_2\}.\end{aligned}$$

Expressiveness

Range restricted RC queries therefore have at least the expressiveness of RA queries. For the other direction, we have the following.

Theorem (Codd)

For every range restricted RC query there is an equivalent RA query. Thus, RA is a relationally complete query language.

Expressiveness

Range restricted RC queries therefore have at least the expressiveness of RA queries. For the other direction, we have the following.

Theorem (Codd)

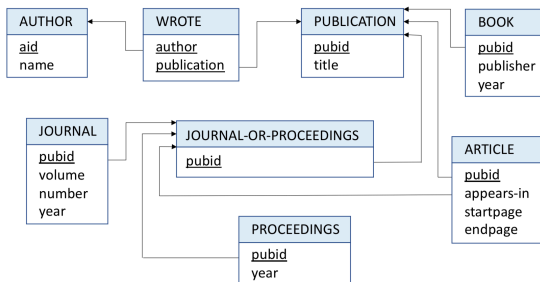
For every range restricted RC query there is an equivalent RA query. Thus, RA is a relationally complete query language.

An outline of translating range restricted RC query conditions to RA queries:

$$\begin{aligned}\text{RCtoRA}(R_i(x_1, \dots, x_k)) &= R_i \\ \text{RCtoRA}(\varphi \wedge x_i = x_j) &= \sigma_{x_i=x_j}(\text{RCtoRA}(\varphi)) \\ \text{RCtoRA}(x_i = c_j) &= x_i = c_j \\ \text{RCtoRA}(\exists x_i. \varphi) &= \pi_{(\text{Fv}(\varphi) - \{x_i\})}(\text{RCtoRA}(\varphi)) \\ \text{RCtoRA}(\varphi_1 \wedge \varphi_2) &= \text{RCtoRA}(\varphi_1) \times \text{RCtoRA}(\varphi_2) \\ \text{RCtoRA}(\varphi_1 \vee \varphi_2) &= \text{RCtoRA}(\varphi_1) \cup \text{RCtoRA}(\varphi_2) \\ \text{RCtoRA}(\varphi_1 \wedge \neg \varphi_2) &= \text{RCtoRA}(\varphi_1) - \text{RCtoRA}(\varphi_2)\end{aligned}$$

1. Must ensure $\text{Fv}(Q_1) \cap \text{Fv}(Q_2) = \emptyset$ for $\wedge$ case; and
2. Need to add a top-level projection to the RA query to map an RC query.

Practice Exercises



1. *What are all publication titles?*
2. *What are the publication titles that are journals or proceedings?*
3. *What are the titles of all books?*
4. *What are the publications without authors?*
5. *What are all the ordered pairs of coauthor names?*
6. *What are all publication titles written by a single author?*

Summary

Relational algebra is the basis for the implementation of SQL.

- ▶ Provides a connection between conceptual and physical level (Module 11).
- ▶ Expresses query execution in (easily) manageable pieces.
- ▶ Allows the use of efficient algorithms/data structures (Module 11).
- ▶ Provides a mechanism for *query optimization* based on logical transformations, including simplifications based on integrity constraints (Module 11).